

Como o código funciona, pasta por pasta

A metodologia do projeto, da organização em módulos ao que cada arquivo lê, faz e entrega. Para quem vai mexer, revisar ou defender o código.

O QUE É O GIROSUS

GiroSUS é um painel de Power BI para planejamento da ocupação de leitos do SUS em Goiás, feito com dado oficial e mensal das internações (SIH/SUS). Não é uma ferramenta de gestão de leitos em tempo real.

USAMOS UMA FERRAMENTA DE LEITO?

Não. O GiroSUS não usa censo hospitalar, sistema de regulação nem cadastro de leitos. Ele mede o uso do leito a partir das internações pagas pelo SUS. O CNES (leitos cadastrados) só foi usado na exploração.

4

camadas de dados

3

fontes (SIH, CNES, IBGE)

15

testes automáticos

6

notebooks

O que tem neste guia

PÁGINA	ASSUNTO
3	A metodologia em 1 página
4	O mapa de pastas e arquivos
5	O fluxo dos dados e quem chama quem
6	As camadas de dados: raw, staging, silver e gold
7	src/config.py: o painel de controle
8	src/extract: ler os arquivos brutos
9	src/transform: padronizar cada fonte
10	src/pipeline.py e src/orchestration: executar tudo
11	tests: os testes de qualidade
12	notebooks: exploração e análise
13	gerar_modelo_bi_girosus.py: a camada do produto
14	Arquivos de apoio: README, dicionário, requirements, .gitignore, .vscode
15	Convenções de código usadas no projeto
16	Como rodar tudo, na ordem
17	Pontos de atenção e dívidas técnicas
18	Glossário técnico
19	Lista dos tratamentos: o que é feito nos dados, por quê e quantas linhas afeta

💡 Como ler

Cada seção explica **o que o arquivo faz, o que ele lê, o que ele entrega e quem usa o que ele entrega**. Os trechos de código são cópias do repositório.

A metodologia em 1 página

O projeto segue um **ETL modular em camadas**: cada fonte de dado tem seu próprio módulo de extração e de transformação, e os dados passam por camadas cada vez mais limpas até virar produto.

1. Camadas de dados

raw (bruto) → staging (saída do pipeline) → silver (validada) → gold (pronta para o produto). É a ideia da “arquitetura medalhão”: cada camada tem um papel e só lê a anterior.

2. Separação por responsabilidade

`extract/` só lê, `transform/` só padroniza, `pipeline.py` só executa na ordem. Nenhum arquivo faz tudo.

3. Um módulo por fonte

SIH, CNES e IBGE têm um arquivo cada em `extract/` e em `transform/`. Mudar uma fonte não mexe nas outras.

4. Configuração centralizada

Caminhos, anos e UF ficam só em `src/config.py`. Os outros arquivos importam de lá, sem caminho escrito à mão.

5. Qualidade testada

O `pytest` confere a staging (nulos, meses, códigos, chaves) e se ela é igual à silver.

6. Regra de negócio fora do pipeline

O pipeline não filtra nada: guarda todas as AIHs. Os filtros (`IDENT = 1`, período, grupo de doença) ficam nos notebooks e no script do GiroSUS.

7. Parquet por dentro, CSV na entrega

Raw, staging e silver em Parquet (compacto, guarda o tipo de cada coluna). A saída do painel (gold) é CSV, o formato que o time de BI usa.

8. Reprodutível

Do `.dbc` ao painel com quatro comandos: converter, pipeline, promover e gerar o modelo. Cada um mostra na tela o que fez.

Resumo em uma frase

Cada pedaço do código faz uma coisa só, lê de uma camada e grava na seguinte; a regra de negócio vive num lugar só, e os testes garantem que a base está certa antes de alguém analisar.

O mapa de pastas e arquivos

```
leitos-sus/
├── src/                                # o código do pipeline (ETL)
│   ├── config.py                     # caminhos, anos e UF: o painel de controle
│   ├── converter_dbc.py              # .dbc do DATASUS → .parquet em data/raw
│   ├── pipeline.py                   # roda extract → transform → grava staging
│   ├── promover_silver.py            # testa a staging e copia para a silver
│   ├── extract/                      # lê os arquivos brutos
│   │   ├── sih.py  cnes.py  ibge.py
│   ├── transform/                   # padroniza cada fonte
│   │   ├── sih.py  cnes.py  populacao.py
│   └── orchestration/
│       └── prefect_flow.py           # o mesmo pipeline, como flow do Prefect
├── tests/                             # testes de qualidade (pytest)
│   ├── conftest.py                  # carrega a staging para os testes
│   ├── test_sih.py  test_cnes.py  test_populacao.py
│   └── test_staging_silver.py        # staging é igual à silver?
├── notebooks/                        # 01-03 exploração · 04 consolidação · 05-06 análise
├── data/
│   ├── raw/                         # arquivos mensais do DATASUS e planilhas do IBGE
│   ├── staging/                     # saída do pipeline (fora do Git)
│   ├── silver/                     # bases validadas, lidas pelas análises
│   └── gold/girosus/                # 10 CSVs do painel (gerados pelo script, fora do Git)
├── gerar_modelo_bi_girosus.py        # cria a camada gold do produto
├── dashboard/                       # material do painel GiroSUS (PDF)
├── docs/                            # dicionário em .csv e este PDF do código
├── requirements.txt                  # bibliotecas e versões
├── .vscode/settings.json             # faz o VS Code usar o .venv
├── .gitignore                       # o que não vai para o Git
└── README.md                        # porta de entrada do projeto
```

<> Código

`src/`, `tests/` e o script do GiroSUS.

📁 Dados

`data/`, separado por camada.

📄 Documentação

README, dicionário e o PDF do painel.

O fluxo dos dados e quem chama quem



<> Quem importa o quê

ARQUIVO	IMPORTA / CHAMA	GRAVA EM
src/pipeline.py	config · extract.sih/cnes/ibge · transform.sih/cnes/populacao	data/staging/*.parquet
src/converter_dbc.py	config · pysus (ExtensionFactory)	data/raw/<fonte>/<ano>/*.parquet
src/promover_silver.py	config · pytest (subprocess) · shutil	data/silver/*.parquet
orchestration/prefect_flow.py	config · pipeline.processar_sih/cnes/populacao	data/staging (via pipeline)
extract/*.py	config (ANOS e pastas de data/raw)	nada: devolvem um DataFrame
transform/*.py	só pandas	nada: devolvem um DataFrame
tests/conftest.py	config (caminhos da staging)	nada: entrega fixtures aos testes
tests/test_staging_silver.py	config · pyarrow	nada: compara staging × silver
notebooks 05 e 06	pandas (lê ../data/silver)	nada (06 também lê data/raw para VAL_UTI)
gerar_modelo_bi_girosus.py	pandas, numpy (lê data/silver e data/raw)	data/gold/girosus/*.csv

💡 Por que o pipeline não grava direto na silver?

Para a silver ser sempre uma versão **conferida**. O pipeline grava na staging e o `promover_silver.py` roda os testes; só se passarem ele copia para a silver. É uma “área de quarentena”.

raw, staging, silver e gold

CAMADA	O QUE TEM	QUEM CRIA	QUEM LÊ	NO GIT?
<code>data/raw</code>	SIH (RDGO*.parquet) e CNES (LTGO*.parquet) por ano; planilhas do IBGE. Todas as colunas como texto	Download manual + <code>src/converter_dbc.py</code>	<code>src/extract</code> · script do GiroSUS (só VAL_UTI)	Sim (menos os .dbc)
<code>data/staging</code>	sih, cnes e populacao_multianual.parquet	<code>src/pipeline.py</code>	pytest	Não
<code>data/silver</code>	As mesmas 3 bases, conferidas	<code>src/promover_silver.py</code>	Notebooks 05 e 06 · script do GiroSUS · <code>test_staging_silver</code>	Sim
<code>data/gold/girosus</code>	10 tabelas do painel em CSV (fatos e dimensões)	<code>gerar_modelo_bi_girosus.py</code>	Power BI	Não (fato em CSV > 250 MB)

O tamanho de cada base (silver)

ARQUIVO	LINHAS	COLUNAS	1 LINHA É
<code>sih_multianual.parquet</code>	2.286.755	23	1 AIH, competências jan/2021 a jun/2026
<code>cnes_multianual.parquet</code>	198.448	12	1 tipo de leito de 1 hospital em 1 mês (jan/2021 a jul/2026)
<code>populacao_multianual.parquet</code>	33.422	10	1 município em 1 ano (2021 a 2026)

✓ Colunas novas chegam pela promoção

O `extract/sih.py` lê SEXO, IDADE, COD_IDADE e GESTAO. Elas chegam na silver quando alguém roda `python -m src.promover_silver`; se a silver for antiga, o teste `test_sih_staging_igual_silver` avisa.

💡 A ideia por trás

Quanto mais para a direita, mais limpo e mais perto do uso. Ninguém analisa o raw direto; ninguém escreve na silver sem passar pelos testes; o Power BI só enxerga o gold.

config.py: o painel de controle

Um arquivo sem funções, só constantes. Todo o resto importa daqui. Se o projeto mudar de estado ou de período, é só mexer aqui.

```
# Projeto
PROJECT_ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = PROJECT_ROOT / "data"
RAW_DIR, STAGING_DIR, SILVER_DIR = DATA_DIR / "raw", ... / "staging", ... / "silver"

# Fontes
SIH_RAW_DIR = RAW_DIR / "sih"
CNES_RAW_DIR = RAW_DIR / "cnes"
IBGE_RAW_DIR = RAW_DIR / "ibge"

# Período e localidade
ANO_INICIAL, ANO_FINAL = 2021, 2026
ANOS = range(ANO_INICIAL, ANO_FINAL + 1)
UF, CODIGO_UF = "GO", "52"

# Saídas: staging e silver de cada fonte
SIH_STAGING_PATH = STAGING_DIR / "sih_multianual.parquet" # e CNES, POPULACAO
SIH_SILVER_PATH = SILVER_DIR / "sih_multianual.parquet" # e CNES, POPULACAO
```

CONSTANTE	QUEM USA	PARA QUÊ
PROJECT_ROOT	todas as outras	Monta caminhos absolutos: o código funciona de qualquer pasta
ANOS	extract/sih.py, extract/cnes.py	Quais pastas de ano ler em data/raw
_RAW_DIR	extract/.py	Onde estão os arquivos brutos
*_STAGING_PATH	pipeline.py, tests	Onde gravar e o que testar
*_SILVER_PATH	test_staging_silver.py	Com o que comparar
UF, CODIGO_UF	(referência)	O recorte é Goiás; os arquivos já vêm filtrados pelo nome (RDGO, LTGO)

💡 Porque Path(__file__) ?

Porque assim o caminho da pasta `data` é calculado a partir do próprio arquivo, e não de onde o terminal está. O script do GiroSUS e os notebooks, que usam caminhos relativos, precisam ser rodados da pasta certa (raiz e notebooks/).

extract/: ler os arquivos brutos

Regra da pasta: só ler e juntar, sem transformar. Cada arquivo devolve um DataFrame com os arquivos mensais empilhados.

extract/sih.py

COLUNAS_SIH

Lista das 22 colunas lidas do arquivo RD (datas, hospital, municípios, dias, UTI, valor, diagnóstico, procedimento, óbito, sexo, idade, gestão...). Ler só o necessário deixa o processo leve.

listar_arquivos_sih()

Para cada ano de `ANOS`, procura `data/raw/sih/<ano>/RDG0*.parquet` e devolve a lista ordenada.

extrair_sih()

Lê cada arquivo com essas colunas, adiciona `ARQUIVO_ORIGEM` (ex.: RDGO2505.dbc) e empilha tudo com `pd.concat`. Se não achar arquivo, para com `FileNotFoundError`.

extract/cnes.py

COLUNAS_CNES

9 colunas do arquivo LT: CNES, município, tipo de unidade, tipo de leito, código do leito, quantidades (existentes, SUS, não SUS) e competência.

listar_arquivos_cnes()

Procura `data/raw/cnes/<ano>/LTG0*.parquet`.

extrair_cnes()

Mesmo padrão do SIH: lê, marca `ARQUIVO_ORIGEM`, empilha.

extract/ibge.py

ARQUIVOS_IBGE

Dicionário ano → planilha e aba: 2021, 2022 (TCU/Censo), 2024, 2025 e 2026. Não existe 2023 (é calculado no transform).

extrair_populacao_ano(ano)

Lê a planilha do ano com `pd.read_excel`, cabeçalho na 2ª linha, códigos como texto (pra não perder zeros à esquerda).

extrair_ibge()

Devolve um dicionário {ano: DataFrame} com todos os anos.

💡 Padrão repetido

SIH e CNES seguem o mesmo molde: `COLUNAS_X + listar_arquivos_x() + extrair_x()`. Isso facilita adicionar uma fonte nova copiando o molde.

transform/: padronizar cada fonte

Recebe o DataFrame do extract e devolve outro, padronizado. Não filtra linhas de negócio: só acerta tipos, nomes e colunas derivadas.

transform/sih.py • 1 FUNÇÃO, 4 LINHAS

transformar_sih(df) : converte DT_INTER de texto para data. O resto fica como veio (texto).

```
df["DT_INTER"] = pd.to_datetime(df["DT_INTER"], errors="raise").dt.date
```

errors="raise" : se tiver data inválida, o pipeline para em vez de esconder o problema.

transform/cnes.py

transformar_cnes(df) : tira o ano e o mês do nome do arquivo (LTG02401 → 2024, mês 1) e converte QT_EXIST, QT_SUS e QT_NSUS para número.

transform/populacao.py • O MAIS COMPLEXO

_padronizar_estimativa() Renomeia as colunas da planilha, monta codigo_ibge_7 (UF + município), limpa a população (tira pontos e notas de rodapé), marca origem e tipo.

_padronizar_2022() Mesma coisa para o arquivo do Censo 2022 (TCU), que tem nome de coluna diferente.

transformar_fontes_oficiais() Junta os anos oficiais (2021, 2022, 2024, 2025, 2026).

_criar_populacao_2023() Calcula 2023 pela média de 2022 e 2024 (interpolação), marcado como "Cálculo próprio com dados IBGE".

transformar_populacao() Junta tudo e ordena por ano e código.

O "_" no começo do nome indica função interna: só é usada dentro do próprio arquivo.

pipeline.py e orchestration/

src/pipeline.py: a ordem de execução

```
def processar_sih():      df = transformar_sih(extrair_sih());
df.to_parquet(SIH_STAGING_PATH)
def processar_cnes():    df = transformar_cnes(extrair_cnes());
df.to_parquet(CNES_STAGING_PATH)
def processar_populacao(): df = transformar_populacao(extrair_ibge());
df.to_parquet(POPULACAO_STAGING_PATH)

def executar_pipeline():
    STAGING_DIR.mkdir(parents=True, exist_ok=True)
    processar_sih(); processar_cnes(); processar_populacao()
```

Cada fonte é uma função `processar_x()` = extrair → transformar → gravar → `informar()` (mostra linhas, colunas e tempo). `executar_pipeline()` cria a pasta de staging e roda as três.

Como rodar: `python -m src.pipeline` (o `-m` roda como módulo, para os imports `from src...` funcionarem).

src/promover_silver.py

Roda os testes de qualidade da staging; só se passarem, copia os 3 arquivos para a silver e roda o teste staging × silver. Se algo falhar, a silver fica como estava. `python -m src.promover_silver`

src/converter_dbc.py

Procura em `data/raw` todo `.dbc` sem `.parquet` ao lado e converte com o PySUS (colunas como texto). `python -m src.converter_dbc`

src/orchestration/prefect_flow.py

O mesmo pipeline, organizado para o **Prefect** (ferramenta de orquestração). Cada `processar_x()` vira uma `@task`, e `executar_fluxo()` é o `@flow`.

```
@task(name="Processar SIH")
def tarefa_sih(): processar_sih()
...
@flow(name="pipeline-leitos-sus")
def executar_fluxo(): tarefa_sih(); tarefa_cnes(); tarefa_populacao()
```

Para que serve: registra cada execução, mostra qual etapa falhou e permite agendar. Roda local, sem servidor: `python -m src.orchestration.prefect_flow`.

💡 Por que existem os dois?

O `pipeline.py` é o jeito simples, sem dependência. O Prefect é a versão “profissional”, com rastreo. Os dois chamam as mesmas funções: a lógica não é duplicada.

tests/: os testes de qualidade

Rodam com `python -m pytest` (o `pytest` sozinho não acha o pacote `src`). Leem a **staging** (não a **silver**) e checam se os dados estão prontos para virar **silver**.

confest.py: as “fixtures”

Carrega cada base da staging uma vez só (`scope="session"`) e entrega para os testes pelos nomes `df_sih` , `df_cnes` e `df_populacao` . Um teste que pede `df_sih` recebe o DataFrame pronto.

ARQUIVO	TESTE	O QUE GARANTE
test_sih.py	cobertura_temporal_sih	66 competências, de 2021-01 a 2026-06
	codigos_municipais_sih_validos	MUNIC_RES e MUNIC_MOV com 6 dígitos
	colunas_criticas_sih_sem_nulos	Datas, AIH, CNES e municípios sem nulos
	datas_sih_validas	DT_INTER e DT_SAIDA viram data válida
test_cnes.py	chave_cnes_sem_duplicados	CNES + tipo + código do leito + competência é único
	quantidade_leitos_consistente	existentes = SUS + não SUS
	cobertura_temporal_cnes	67 competências, de 2021-01 a 2026-07
	colunas_criticas_cnes_sem_nulos	Colunas-chave sem nulos
test_populacao.py	chave_populacao_sem_duplicados	Município + ano é único
	codigos_ibge_validos	Código com 7 dígitos
	cobertura_temporal_populacao	Anos 2021 a 2026
	proveniencia_populacao_2023	2023: 5.570 municípios, marcado como cálculo próprio
test_staging_silver.py	sih_staging_igual_silver	Mesmas linhas, mesmo esquema e mesmos valores, coluna a coluna (com pyarrow)
	cnes_staging_igual_silver	
	populacao_staging_igual_silver	

Dois tipos de teste

Qualidade (sih, cnes, populacao): o dado faz sentido? **Regressão** (staging × silver): o dado mudou sem ninguém saber? Se o segundo falhar de propósito (ex.: colunas novas), a solução é promover a staging.

notebooks/: exploração e análise

NOTEBOOK	PAPEL	LÊ	OBSERVAÇÃO
01_exploracao_sih	Conhecer o SIH: tamanho, tipos, nulos, duplicidades	.dbc (PySUS)	Não roda num clone novo (os .dbc não vão pro Git)
02_exploracao_cnes	Conhecer o CNES de leitos	.dbc (PySUS)	Idem
03_exploracao_ibge	Conhecer as planilhas de população	data/raw/ibge	Roda normal
04_consolidacao_multianual	1ª versão da consolidação que hoje o pipeline faz	.dbc	A última célula grava na silver: não rodar sem necessidade
05_analise_integrada	Cruza SIH, CNES e população: demanda, capacidade, fluxo entre municípios	data/silver	Roda normal
06_girosus_ocupacao_leitos	Análise do GiroSUS: as 34 perguntas do painel, safra 2025	data/silver (+ data/raw para VAL_UTI)	Mesmos números do script e do PDF

Regra de ouro

Notebooks de análise (05 e 06) leem só a **silver**. Assim todo mundo analisa a mesma versão conferida dos dados.

Caminhos relativos

Os notebooks usam `../data/silver`. O `.vscode/settings.json` faz o VS Code rodar cada notebook a partir da pasta dele (`jupyter.notebookFileRoot`).

Estrutura do notebook 06

1 configuração → 2 carregamento → 3 regras e funil (467.902 → 457.403) → 4 colunas do produto (leito-dia, grupo, tempo típico) → 5 a 12 os três níveis de perguntas → 13 curiosidades de pagamento → 14 resumo para a venda → 15 conclusão e limites. Usa funções pequenas (`classificar_grupo`, `resumo`, `censo_diario`) para não repetir código.

O script da camada gold

Fica na raiz porque não faz parte do pipeline de ingestão: é a camada do **produto**. Lê a silver, aplica as regras do GiroSUS e grava as tabelas do Power BI.

<code>classificar_grupo(cid)</code>	1ª letra do CID → grupo de doença (J = Respiratório, D00–D48 = Câncer...).
<code>carregar_sih()</code>	Lê a silver e já filtra IDENT = 1.
<code>carregar_valor_uti()</code>	Lê só N_AIH, competência e VAL_UTI dos brutos (a silver ainda não tem). Sem os brutos, avisa e segue.
<code>nomes_municipios()</code>	Tabela código → nome e UF, a partir da base do IBGE.
<code>montar_fato_internacoes()</code>	Renomeia colunas, converte tipos, calcula tempo_típico (mediana por safra e procedimento), dias_acima_típico, faixa_duracao, fl_fora_municipio, fl_uti e junta valor_uti.
<code>montar_fato_ocupacao()</code>	Expande cada internação em dias (com numpy, sem laço) e conta, por hospital, dia e grupo: internados, entradas e altas.
<code>montar_dim_tempo()</code>	Calendário diário de 2020 a 2026 com nomes de mês e dia.
<code>main()</code>	Monta as 10 tabelas, apaga .parquet antigos, grava os 10 .csv ("," e ",") em data/gold/girosus e imprime a conferência da safra 2025.
<code>conferir_saida()</code>	Confere se os 10 CSVs existem com todas as colunas e sem nenhum vazio; se não, para com ERRO.

```
# a regra do tempo típico, numa linha
fato["tempo_típico"] = fato.groupby(["safra", "proc_rea"])["leito_dias"].transform("median")
fato["dias_acima_típico"] = (fato["leito_dias"] - fato["tempo_típico"]).clip(lower=0)
```

SAI EM DATA/GOLD/GIROSUS

fato_internacoes

fato_ocupacao_diaria

dim_tempo · dim_diagnostico · dim_procedimento · dim_hospital

dim_municipio_residencia · dim_municipio_atendimento

sazonalidade_mensal · tendencia_anual (tabelas de apoio, sem relacionamento)

✓ Autoconferência

No fim, imprime os totais esperados da safra 2025 (457.403 internações, 1.835.227 leitos-dia, R\$ 726,1 mi, 43,9%). Se não bater com o notebook 06 e o PDF, algo mudou na silver.

README, dicionário e configurações

ARQUIVO	PARA QUE SERVE
README.md	Porta de entrada: o produto, onde está cada coisa, como rodar, fluxo, regras, modelo do Power BI, dicionário de dados, notebooks, testes (com o roteiro para conferir o projeto inteiro), como atualizar, problemas, contribuição e status do projeto.
docs/dicionário/dicionario_dados.csv	As 93 colunas (silver e gold) em planilha, para filtrar. A versão para ler está no README.
dashboard/GiroSUS_painel_ocupacao_leitos.pdf	Material do produto: pitch, páginas do painel, perguntas, guia do analista e método.
requirements.txt	pandas 2.3.3 · pyarrow 25.0.1 · pysus 2.11.2 · openpyxl · xlrd · prefect 3.8.6 · pytest 9.1.1 · ipykernel<7
.vscode/settings.json	Faz o VS Code usar o Python do <code>.venv</code> , ativar o ambiente no terminal e rodar notebooks a partir da pasta deles.
.gitignore	Deixa fora do Git: <code>*.dbc</code> e <code>*.dbf</code> , microdados grandes do Censo, <code>data/staging/</code> , <code>data/gold/girosus/</code> , caches do Python e do pytest, e o <code>.venv/</code> .

☼ Para que serve cada biblioteca

BIBLIOTECA	USO NO PROJETO
pandas	Toda a manipulação de tabelas
pyarrow	Ler e gravar Parquet; comparar staging × silver
pysus	Ler e converter os .dbc do DATASUS (converter_dbc e notebooks 01, 02 e 04)
openpyxl · xlrd	Ler as planilhas .xlsx e .xls do IBGE
prefect	Orquestração opcional do pipeline
pytest	Testes de qualidade
ipykernel (<7)	Rodar notebooks no VS Code (a versão 7 trava a extensão Jupyter)

As convenções do código

CONVENÇÃO	COMO APARECE	POR QUÊ
Nomes em português	<code>extrair_sih</code> , <code>transformar_cnes</code> , <code>df_populacao</code>	O time e o domínio (SUS) são brasileiros; o código lê como o negócio fala
snake_case para funções e variáveis	<code>listar_arquivos_sih</code>	Padrão do Python (PEP 8)
MAIÚSCULAS para constantes	<code>COLUNAS_SIH</code> , <code>SIH_STAGING_PATH</code>	Deixa claro o que não muda durante a execução
Verbo no nome da função	<code>extrair_</code> , <code>transformar_</code> , <code>processar_</code> , <code>montar_</code>	O nome diz a etapa
"_" para função interna	<code>_padronizar_2022</code>	Sinaliza que só é usada dentro do arquivo
Type hints	<code>-> pd.DataFrame</code> , <code>-> list[Path]</code>	Mostra o que entra e sai sem ler o corpo
pathlib em vez de texto	<code>RAW_DIR / "sih"</code>	Funciona em Windows e Linux
Comentário de 1 linha por bloco	<code># Carrega e consolida os arquivos do SIH</code>	Explica a intenção, não repete o código
Códigos como texto	<code>str.zfill(6)</code> , <code>dtype={"COD. MUNIC": str}</code>	Não perder zeros à esquerda (CNES 0965324)
Erro explícito	<code>raise FileNotFoundError(...)</code> , <code>errors="raise"</code>	Parar cedo em vez de gerar dado errado
Rastreabilidade	coluna <code>ARQUIVO_ORIGEM</code>	Saber de qual arquivo mensal veio cada linha
Nomes de arquivo com o conteúdo	<code>sih_multianual.parquet</code> , <code>test_staging_silver.py</code>	O nome já diz o que tem dentro
Notebooks numerados	<code>01_</code> a <code>06_</code>	A ordem é a ordem do raciocínio

Como rodar tudo, na ordem

No Git Bash, na raiz do projeto.

```
# 0. Uma vez: criar e ativar o ambiente, instalar dependências
py -3.12 -m venv .venv
source .venv/Scripts/activate          # aparece (.venv) no começo da linha
pip install -r requirements.txt

# 1. Só quando entrar mês novo: .dbc baixado → .parquet
python -m src.converter_dbc

# 2. Reconstruir a staging (mostra cada base gravada)
python -m src.pipeline                  # ou: python -m src.orchestration.pflect_flow

# 3. Testar e promover staging → silver (só copia se passar)
python -m src.promover_silver

# 4. (Opcional) Rodar todos os testes de novo: 15 passed
python -m pytest

# 5. Gerar as tabelas do painel GiroSUS (10 CSVs + conferência)
python gerar_modelo_bi_girosus.py
```

QUERO...	RODO
Só atualizar o Power BI com a silver atual	Passo 5
Incluir um mês novo do SIH/CNES	Baixar o .dbc → passos 1, 2, 3 e 5 (e ajustar os testes de cobertura)
Estudar os números	Abrir o notebook 06_girosus_ocupacao_leitos e “Run All”
Conferir que tudo funciona	Passos 1 a 5 e o roteiro “Como conferir que o projeto inteiro funciona” do README

⚠ Se o comando não mostrar nada

No Windows, `python` sem o `.venv` ativo pode chamar um atalho da Microsoft Store que não faz nada. Ative o ambiente ou use `.venv/Scripts/python.exe` no lugar de `python`.

💡 Avisos amarelos (DeprecationWarning)

Vêm das bibliotecas (dateutil, NumPy), não do projeto, e não mudam nenhum número. Só é problema se aparecer `ERROR` ou `FAILED`.

Dívidas técnicas e pontos de atenção

O QUÊ	ONDE	SUGESTÃO
1 Download dos .dbc ainda é manual	—	Automatizar o download no <code>converter_dbc.py</code>
2 Notebooks 01, 02 e 04 dependem dos .dbc	notebooks/	Ler os .parquet de data/raw
3 Notebook 04 grava na silver com DT_INTER como texto	notebooks/04	Não rodar a última célula / remover a gravação
4 VAL_UTI não está na silver (o script lê do raw)	extract/sih.py	Adicionar VAL_UTI em COLUNAS_SIH
5 Regras de negócio duplicadas (notebook 06 e script)	notebooks/06 e script	Mover regras para um módulo (ex.: <code>src/girosus.py</code>) e importar nos dois

Como falar disso numa avaliação

“O projeto tem as dívidas mapeadas. Já resolvemos a promoção automática, as mensagens do pipeline e a conversão dos .dbc; as que restam são de organização e não mudam os números.”

Glossário técnico

TERMO	EM PORTUGUÊS CLARO
ETL	Extract, Transform, Load: extrair, transformar e carregar. É o que o src/ faz.
Pipeline	Sequência automática de etapas: aqui, extract → transform → gravar.
Camada (raw, staging, silver, gold)	Nível de tratamento do dado. Quanto mais para o gold, mais pronto para uso.
Arquitetura medalhão	Nome comum para organizar dados em camadas bronze (bruto), silver (limpo) e gold (pronto).
Módulo	Um arquivo .py que pode ser importado por outro (ex.: src.extract.sih).
Parquet	Formato de arquivo colunar, compacto, que guarda o tipo de cada coluna.
DataFrame	A tabela do pandas: linhas e colunas na memória.
Fixture	No pytest, um dado preparado uma vez e entregue aos testes que pedem por ele.
Teste de regressão	Confere se algo mudou sem ninguém querer (staging × silver).
Orquestração (Prefect)	Ferramenta que executa, registra e agenda as etapas do pipeline.
Ambiente virtual (.venv)	Uma instalação de Python só do projeto, com as versões do requirements.
Modelo estrela	Tabela fato no centro ligada a dimensões; é como o gold está organizado para o Power BI.
Fato / dimensão	Fato = o que aconteceu (internação). Dimensão = como filtrar (tempo, hospital, doença).
Type hint	Anotação do tipo que entra e sai de uma função (-> pd.DataFrame).
Competência	Mês em que o hospital apresentou a internação para pagamento; organiza os arquivos do DATASUS.

O que é feito nos dados, linha a linha

Todos os tratamentos, do arquivo do DATASUS até o CSV do Power BI: o que é feito, por quê, onde está no código e quantas linhas afeta. Números da execução de 30/09/2026.

#	TRATAMENTO	POR QUÊ	ONDE ESTÁ NO CÓDIGO	LINHAS AFETADAS
1	Conversão de .dbc para .parquet	O DATASUS publica em .dbc, um formato compactado antigo. O Parquet é leve e guarda o tipo de cada coluna. Tudo entra como texto, sem mudar nenhum valor.	<code>src/converter_dbc.py</code> (PySUS)	66 arquivos do SIH (2.286.755 registros) e 67 do CNES (198.448 linhas)
2	Datas e números no tipo certo	O arquivo chega todo em texto. Somar dias e valores e filtrar por data exigem o tipo certo.	<code>src/transform/sih.py</code> (DT_INTER), <code>src/transform/cnes.py</code> (quantidades de leitos), <code>gerar_modelo_bi_girosus.py</code> (DT_SAIDA, DIAS_PERM, VAL_TOT, UTI)	2.286.755 AIHs do SIH e 198.448 linhas do CNES
3	Ano e mês tirados do nome do arquivo	Cada arquivo de leitos é de um mês (LTGO2401 = jan/2024). O nome garante que cada linha fique com o mês do arquivo de onde veio.	<code>src/transform/cnes.py</code>	198.448 linhas do CNES
4	Códigos com zero à esquerda preservados	CNES e município são códigos, não números: 0965324 não pode virar 965324, senão não liga com as outras tabelas.	Leitura como texto em <code>src/extract/</code> ; <code>zfill</code> em <code>gerar_modelo_bi_girosus.py</code> ; consultas <code>power_query/*.pq</code> importam como Texto	165.522 AIHs com CNES começando em 0 (38.662 na safra 2025) e 7.043 linhas do CNES
5	Só AIH nova (IDENT = 1)	A AIH de continuação (IDENT = 5) é a mesma internação cobrada em partes. Contar as duas duplicaria a internação.	<code>gerar_modelo_bi_girosus.py</code> · <code>carregar_sih()</code>	2.286.755 → 2.235.030 (saem 51.725)
6	Safra 2025 pela competência	O painel olha o que foi apresentado ao SUS para pagamento de janeiro a dezembro de 2025.	<code>gerar_modelo_bi_girosus.py</code> · <code>main()</code>	2.235.030 → 457.403
7	Leito-dia, tempo típico, dias acima do típico e grupo pelo CID	Regras do produto: medir a ocupação em dias, comparar com a mediana do mesmo procedimento e agrupar as doenças pela primeira letra do CID-10.	<code>montar_fato_internacoes()</code> e <code>classificar_grupo()</code>	457.403 internações; 163.861 com dias acima do típico
8	UTI e acompanhante tirados do arquivo bruto	VAL_UTI e DIAR_ACOM ainda não estão na silver. Sem o arquivo bruto, o script para com erro em vez de gerar coluna vazia.	<code>carregar_complementos_raw()</code>	457.403 internações ligadas; 41.880 com valor de UTI e 236.063 com diária de acompanhante
9	Testes de vazios e duplicados	Garantir que a base está certa antes de qualquer análise. A silver só é atualizada se todos os testes passarem.	<code>tests/</code> (15 testes) e <code>src/promover_silver.py</code>	Toda a staging: 2.286.755 (SIH), 198.448 (CNES) e 33.422 (população)
10	Trava contra colunas vazias	Nenhuma tabela do painel pode sair com célula vazia, como já aconteceu com <code>nome_hospital</code> , que o SIH não tem. A trava também para se essa coluna voltar.	<code>conferir_saida()</code>	As 10 tabelas do painel; hoje 0 células vazias